



# UNIVERSIDAD DE GRANADA

Periféricos y Dispositivos de Interfaz Humana

Práctica 1

Pablo Soriano Fleitas

## gotoxy()

Colocar el cursor en una posición determinada

```
1  #include <stdio.h>
2  #include <dos.h>
3
4  void xy(int x, int y){
5      union REGS inregs, outregs;
6      inregs.h.ah = 0x02;
7      inregs.h.bh = 0x00;
8      inregs.h.dh = y;
9      inregs.h.dl = x;
10     int86(0x10,&inregs,&outregs);
11     return;
12 }
13
14 int main(){
15
16     xy(15,7);
17
18     return 0;
19 }
20
```

Reutilizo la función xy que se emplea en el archivo gotoxy en el que se desplaza el cursor y se dibuja el caracter "\*", aunque en este caso simplemente desplazo el cursor a la posición indicada.

Ejemplo de ejecución:

```
C:\P1EJ>
C:\P1EJ>
C:\P1EJ>
C:\P1EJ>
C:\P1EJ>_
C:\P1EJ>
C:\P1EJ>
C:\P1EJ>
C:\P1EJ>
C:\P1EJ>
C:\P1EJ>
C:\P1EJ>
C:\P1EJ>EJ1GTXY.EXE
```

## setcursortype():

fixar el aspecto del cursor, debe admitir tres valores: INVISIBLE, NORMAL y GRUESO.

```
1  #include <stdio.h>
2  #include <dos.h>
3
4  struct pareja {
5      int a;
6      int b;
7  };
8
9  struct pareja INVISIBLE = {0x20,0x00};
10 struct pareja NORMAL = {0x06,0x07};
11 struct pareja GRUESO = {0x04,0x07};
12
13 void mi_pausa(){
14     union REGS inregs, outregs;
15     inregs.h.ah = 0x00;
16     int86(0x21, &inregs, &outregs);
17 }
18
19 void cursortype(struct pareja modo){
20     union REGS inregs, outregs;
21     inregs.h.ah = 0x01;
22     inregs.h.ch = modo.a;
23     inregs.h.cl = modo.b;
24     int86(0x10, &inregs, &outregs);
25     return;
26 }
```

```
28  int main(){
29
30     cursortype(INVISIBLE);
31
32     printf("\nPulsa para aumentar el grosor del cursor\n");
33     mi_pausa();
34
35     cursortype(GRUESO);
36
37     printf("\nPulsa para volver al cursor normal\n");
38     mi_pausa();
39
40     cursortype(NORMAL);
41
42     return 0;
43 }
```

Creo la función cursortype, con el número de función 1, en la que cambio los valores de ch y cl según la pareja de valores que paso como argumento. Para ello creo el struct pareja, donde el primer valor es el de ch, que es la línea inicial del cursor, y el segundo es el de cl, que es la línea final del cursor.

Creo los pares Invisible, donde el 6 bit de CH lo pongo a 1; Normal, manteniendo el tamaño base del cursor; y Grueso, aumentando el tamaño del cursor de manera perceptible.

Finalmente, comienzo pasando el cursor a invisible, para que luego se haga más grueso, y finalmente devolverlo a su tamaño original, de tal forma que se pueda ver que funciona correctamente con los tres valores que permite.

Ejemplo de ejecución:

```
C:\>
C:\>
C:\>
C:\>cd P1EJ
C:\P1EJ>EJ1SCTP.EXE
Pulsa para aumentar el grosor del cursor
```

```
C:\P1EJ>EJ1SCTP.EXE
Pulsa para aumentar el grosor del cursor
Pulsa para volver al cursor normal
```

```
C:\>cd P1EJ
C:\P1EJ>EJ1SCTP.EXE
Pulsa para aumentar el grosor del cursor
Pulsa para volver al cursor normal
C:\P1EJ>
```

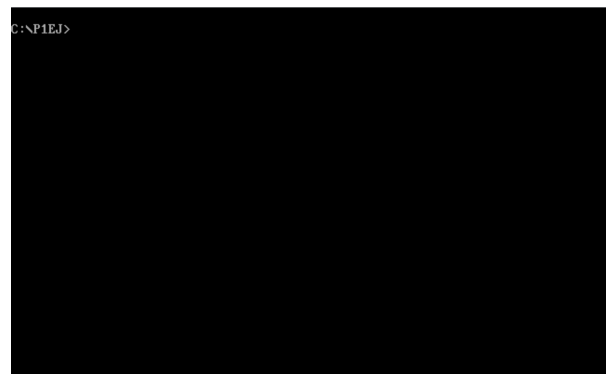
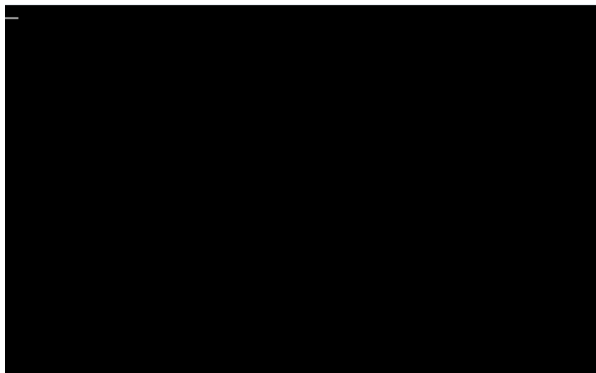
## setvideomode():

fija el modo de video deseado

```
1  #include <dos.h>
2
3  #define BYTE unsigned char
4
5  BYTE MODOTEXTO = 3;
6  BYTE MODOTEXTOGR = 1;
7
8  void pausa(){
9      union REGS inregs, outregs;
10     inregs.h.ah = 0x00;
11     int86(0x16, &inregs, &outregs);
12 }
13
14 void modovideo(BYTE modo){
15     union REGS inregs, outregs;
16     inregs.h.al = modo;
17     inregs.h.ah = 0x00;
18     int86(0x10, &inregs, &outregs);
19 }
20
21 int main(){
22
23     modovideo(MODOTEXTOGR); //gráfico
24
25     pausa();
26
27     modovideo(MODOTEXTO); //texto
28
29     return 0;
30 }
```

Empleo la función modo vídeo, en la que modifíco el valor de “al” según el modo de vídeo que quiera. En este caso empleo 1 para pasar al modo de resolución 40x25 y 3 para volver al modo texto normal.

Ejemplo de ejecución:



## getvideomode():

obtiene el modo de video actual

```
1  #include <stdio.h>
2  #include <dos.h>
3
4  void mi_pausa(){
5      union REGS inregs, outregs;
6      inregs.h.ah = 8;
7      int86(0x21, &inregs, &outregs);
8  }
9
10 int mi_getvideomode(){
11     union REGS inregs, outregs;
12     int mode;
13
14     inregs.h.ah = 0x0F;
15     int86(0x10, &inregs, &outregs);
16
17     mode = outregs.h.al;
18     return mode;
19 }
20
21 int main(){
22     int modo = mi_getvideomode();
23     printf("Modo de video actual: %d\n", modo);
24
25     mi_pausa();
26
27     return 0;
28 }
```

Creo la función getvideomode con el número de función Fh para obtener el modo de vídeo actual, el cual se encuentra almacenado en "al"

Ejemplo de ejecución:

```
C:\P1EJ>
C:\P1EJ>EJ1GUIDM.EXE
Modo de video actual: 3
```

## textcolor():

modifica el color de primer plano con que se mostrarán los caracteres

```
int mi_getchar(){
    union REGS inregs, outregs;
    int character;

    inregs.h.ah = 0x08;
    int86(0x21, &inregs, &outregs);

    character = outregs.h.al;
    return character;
}

void put_char(char c){
    union REGS inregs, outregs;
    inregs.h.ah = 2;
    inregs.h.dl = c;
    int86(0x21, &inregs, &outregs);
}

int leer_numero() {
    int num = 0;
    char c;

    while(1) {
        c = mi_getchar();
        put_char(c);

        if(c == 13) // ENTER
            break;

        if(c >= '0' && c <= '9') {
            num = num * 10 + (c - '0');
        }
    }

    return num;
}
```

```
void avanzar(){
    union REGS inregs, outregs;
    unsigned char fila, col;

    inregs.h.ah = 0x03;
    inregs.h.bh = 0;
    int86(0x10, &inregs, &outregs);

    fila = outregs.h.dh;
    col = outregs.h.dl;

    col++;

    inregs.h.ah = 0x02;
    inregs.h.dh = fila;
    inregs.h.dl = col;
    int86(0x10, &inregs, &outregs);
}
```

```
unsigned char leer_color(){
    int c = leer_numero();

    if(c >= 0 && c <= 15) {
        return (unsigned char)c;
    } else {
        printf("Color inválido. Usando color por defecto (BLANCO).\n");
        return 7; // color blanco por defecto
    }
}

void escribir_char_con_color(char c, unsigned char color){
    union REGS inregs, outregs;
    inregs.h.ah = 0x09;
    inregs.h.al = c;
    inregs.h.bl = 0 | color;
    inregs.h.bh = 0x00;
    inregs.x.cx = 1;
    int86(0x10, &inregs, &outregs);
    return;
}
```

```
int main(){
    unsigned char color;
    char c;
    int num;

    printf("Indique el color con el que desea escribir (0-15): ");
    color = leer_color();

    printf("\n");

    printf("Puede comenzar a escribir (presione ENTER para finalizar)...");
    while(1) {
        c = mi_getchar();

        if(c == 13) // ENTER
            break;

        escribir_char_con_color(c, color);
        avanzar();
    }

    return 0;
}
```

Para este caso reutilizo las funciones `mi_char` y `put_char`, así como `escribir_char_con_color`, la cual utilizo en otro caso igualmente para leer del teclado caracteres y mostrarlos por pantalla con el color seleccionado. A su vez reutilizo la función leer número que he creado en otro ejemplo e implemento la de leer color, la cual la usa y en la que me aseguro que el valor seleccionado para el color sea válido. Finalmente también creo la función `avanzar` para obtener las coordenadas actuales del cursor y modificar simplemente la coordenada de x en 1, para poder ir escribiendo sin que se vaya sobreescibiendo el texto sobre el mismo pixel.

Una vez implementadas las funciones solicito por pantalla el valor del color en el que se desea escribir y finalmente voy obteniendo los caracteres de las teclas que se van pulsando y las muestro con el color indicado por pantalla y voy avanzando con cada iteración hasta que el usuario pulse enter.

Ejemplo de ejecución:

```
C:\P1EJ>ej1txcol
Indique el color con el que desea escribir (0-15): 2
Puede comenzar a escribir (presione ENTER para finalizar)...
Efectivamente me permite escribir a color
```

## textbackground():

modifica el color de fondo con que se mostrarán los caracteres

```
#include <stdio.h>
#include <dos.h>

int mi_getchar(){
    union REGS inregs, outregs;
    int character;

    inregs.h.ah = 0x08;
    int86(0x21, &inregs, &outregs);

    character = outregs.h.al;
    return character;
}

void put_char(char c){
    union REGS inregs, outregs;
    inregs.h.ah = 2;
    inregs.h.dl = c;
    int86(0x21, &inregs, &outregs);
}

int leer_numero() {
    int num = 0;
    char c;

    while(1) {
        c = mi_getchar();
        put_char(c);

        if(c == 13) // ENTER
            break;

        if(c >= '0' && c <= '9') {
            num = num * 10 + (c - '0');
        }
    }

    return num;
}

unsigned char leer_color(){
    int c = leer_numero();

    if(c >= 0 && c <= 15) {
        return (unsigned char)c;
    } else {
        printf("Color inválido. Usando color por defecto (BLANCO).\n");
        return 7; // color blanco por defecto
    }
}
```

```
void escribir_char_con_color(char c, unsigned char color){
    union REGS inregs, outregs;
    inregs.h.ah = 0x09;
    inregs.h.al = c;
    inregs.h.bl = color << 4 | 0;
    inregs.h.bh = 0x00;
    inregs.x.cx = 1;
    int86(0x10, &inregs, &outregs);
    return;
}

void avanzar(){
    union REGS inregs, outregs;
    unsigned char fila, col;

    inregs.h.ah = 0x03;
    inregs.h.bh = 0;
    int86(0x10, &inregs, &outregs);

    fila = outregs.h.dh;
    col = outregs.h.dl;

    col++;

    inregs.h.ah = 0x02;
    inregs.h.dh = fila;
    inregs.h.dl = col;
    int86(0x10, &inregs, &outregs);
}

int main(){
    unsigned char color;
    char c;
    int num;

    printf("Indique el color del fondo con el que desea escribir (0-15): ");
    color = leer_color();

    printf("\n");

    printf("Puede comenzar a escribir (presione ENTER para finalizar)... \n");
    while(1) {
        c = mi_getchar();

        if(c == 13) // ENTER
            break;

        escribir_char_con_color(c, color);
        avanzar();
    }
}
```

En este caso lo que hago es que mantengo las mismas funciones que en el ejercicio anterior, aunque ahora a la hora de escribir el caracter con color cambio los valores del color de fondo en lugar de los valores del color del propio texto como en el caso anterior.

Ejemplo de ejecución:

```
C:\P1EJ>ej1bgcol
Indique el color del fondo con el que desea escribir (0-15): 4
Puede comenzar a escribir (presione ENTER para finalizar)...
hola mundo asi es como se puede ver que el fondo se cambia sin problemaaaSs
```

## clrscr():

borra toda la pantalla

```
#include <dos.h>

#define BYTE unsigned char

BYTE MODOTEXTO = 3;
BYTE MODOTEXTOGR = 1;

void mi_getchar(){
    union REGS inregs, outregs;
    //int character;

    inregs.h.ah = 1;
    int86(0x21, &inregs, &outregs);

    //caracter = outregs.h.al;
    //return character;
}

void modovideo(BYTE modo){
    union REGS inregs, outregs;
    inregs.h.al = modo;
    inregs.h.ah = 0x00;
    int86(0x10, &inregs, &outregs);
}

int main(){

    printf("\nPulsa una tecla para borrar la pantalla... ");

    mi_getchar();

    modovideo(MODOTEXTOGR); //gráfico

    modovideo(MODOTEXTO); //texto

    return 0;
}
```

Empleo la función modovideo para cambiar a modo gráfico y volver a modo texto, puesto que al regresar a modo texto borra toda la pantalla.

Ejemplo de ejecución:

```
C:\P1EJ>EJ1CLS.EXE
Pulsa una tecla para borrar la pantalla... _
```

```
C:\P1EJ>S_
```



## cputchar():

escribe un carácter en pantalla con el color indicado actualmente

```
#include <stdio.h>
#include <dos.h>

unsigned char ctexto=1;

int mi_getchar(){
    union REGS inregs, outregs;
    int character;

    inregs.h.ah = 1;
    int86(0x21, &inregs, &outregs);

    character = outregs.h.al;
    return character;
}

void escribir_char_con_color(char c){
    union REGS inregs, outregs;
    inregs.h.ah = 0x09;
    inregs.h.al = c;
    inregs.h.bl = 0 | ctexto;
    inregs.h.bh = 0x00;
    inregs.x.cx = 1;
    int86(0x10, &inregs, &outregs);
    return;
}

int main(){
    int tmp;
    printf("\nIndique el caracter a imprimir: ");
    tmp = mi_getchar();

    printf("\nIndique el color del texto (0-15): ");
    ctexto = mi_getchar() - '0'; // Lee el color del texto
    printf("\nImprimiendo caracter con color... \n");
    escribir_char_con_color((char)tmp);
}
```

Empleo la función `mi_getchar()` para obtener el caracter que se desea mostrar por pantalla con el color cambiado, a continuación vuelvo a usar esa función para obtener el color al que se quiere mostrar el caracter, convirtiendo debidamente el valor ASCII del número a valor numérico para que la siguiente función lo entienda correctamente, y finalmente empleo la función `excribir_char_con_color` para mostrar por pantalla el caracter con el color indicado.

Ejemplo de ejecución:

```
C:\P1EJ>EJ1ELC.EXE

Indique el caracter a imprimir: s
Indique el color del texto (0-15): 4
Imprimiendo caracter con color...
s
```

## getche():

obtiene un carácter de teclado y lo muestra en pantalla

```
#include <stdio.h>
#include <dos.h>

int mi_getchar(){
    union REGS inregs, outregs;
    int character;

    inregs.h.ah = 0x08;
    int86(0x21, &inregs, &outregs);

    character = outregs.h.al;
    return character;
}

void mi_putchar(char c){
    union REGS inregs, outregs;

    inregs.h.ah = 2;
    inregs.h.dl = c;
    int86(0x21, &inregs, &outregs);
}

int main(){
    int tmp;

    printf("\nIndique el caracter a imprimir... ");
    tmp = mi_getchar();

    printf("\nHa seleccionado el siguiente caracter: ");
    mi_putchar((char)tmp);

    return 0;
}
```

Empleo la función `mi_getchar` para obtener el caracter que se quiere mostrar por pantalla, finalmente empleo la función `mi_putchar` para mostrar ese caracter por pantalla.

C:\P1EJ>ej1g.c

Indique el caracter a imprimir...

Ha seleccionado el siguiente caracter: s

## pixel():

dibujar un pixel en modo gráfico (la función recibirá la coordenada x,y y el color del punto)

```
#include <dos.h>

#define BYTE unsigned char

BYTE MODOTEXTO = 3;
BYTE MODOGRAFICO = 4;

void pausa(){
    union REGS inregs, outregs;
    inregs.h.ah = 0x00;
    int86(0x16, &inregs, &outregs);
}

int mi_getchar(){
    union REGS inregs, outregs;
    int character;

    inregs.h.ah = 1;
    int86(0x21, &inregs, &outregs);

    character = outregs.h.al;
    return character;
}
```

```
/* ===== */
int main(){
    int i;
    int valx , valy ;
    int color;

    printf("\nIndique el valor de la coordenada x: ");
    valx = leer_numero();

    printf("\nIndique el valor de la coordenada y: ");
    valy = leer_numero();

    printf("\nIndique el color (0-15): ");
    color = leer_numero();

    modovideo(MODOGRAFICO); //gráfico

    pixel(valx, valy, (BYTE)color); //poner un pixel en la coordenada (valx,valy) de color "color"

    pausa();
    modovideo(MODOTEXTO); //texto

    return 0;
}
```

```
int leer_numero() {
    int num = 0;
    char c;

    while(1) {
        c = mi_getchar();

        if(c == 13) // ENTER
            break;

        if(c >= '0' && c <= '9') {
            num = num * 10 + (c - '0');
        }
    }

    return num;
}

void modovideo(BYTE modo){
    union REGS inregs, outregs;
    inregs.h.al = modo;
    inregs.h.ah = 0x00;
    int86(0x10, &inregs, &outregs);
}

void pixel(int x, int y, BYTE C){
    union REGS inregs, outregs;
    inregs.x.cx = x;
    inregs.x.dx = y;
    inregs.h.al = C;
    inregs.h.ah = 0x0C;
    int86(0x10, &inregs, &outregs);
}
```

Empleo la función `mi_getchar` para obtener como entrada de teclado los valores de las coordenadas donde se quiera dibujar el pixel, así como el valor del Byte del color en el que se quiere mostrar el pixel, sin embargo, añado una nueva función a la que llamo `leer número`, puesto que `mi_getchar` solo me permite leer del teclado caracteres de uno en uno, y de esta manera puedo leer números que sean más grandes que del 0 al 9, detectando cuando se pulse enter para finalizar la lectura de caracteres, a su vez los transformo de carácter a int antes de devolverlos. A continuación empleo la función `modovideo` para cambiar entre el Modo gráfico y el Modo texto, usando la función `pixel` que dibuja el pixel en las coordenadas “x” e “y” dadas con el color “color”, una vez y entro en el modo gráfico. Finalmente uso la función `pausa` para esperar a que se pulse cualquier tecla para volver al modo texto.

Ejemplo de ejecución:

```
C:\P1EJ>ej1dbp
```

```
Indique el valor de la coordenada x: 100
```

```
Indique el valor de la coordenada y: 100
```

```
Indique el color (0-15): 2
```



## PARA AMPLIAR

- 1.** Implementar una función que permita dibujar un recuadro en la pantalla en modo texto. Recibirá como parámetros las coordenadas superior izquierda e inferior derecha del recuadro, el color de primer plano y el color de fondo.
- 2.** Implementar en lenguaje C un programa que establezca modo gráfico CGA (modo=4) para crear dibujos sencillos en pantalla.
- 3.** Implementar un programa sencillo que realice un dibujo sencillo de tipo “ascii art”. En el ANEXO al final de este guión se proponen algunos diseños.